Projeto e Análise de Algoritmos

Aulas 1 e 2

Universidade Federal do Amazonas
Instituto de Computação

A idade das coisas

- Universo: Estima-se 13 bilhões (pode ser mais, pode ser menos...).
- Terra: 4,54 bilhões de anos (pode ser menos, pode ser mais...)
- Sol: 5 bilhões de anos e mais 3 a 6 bilhões de vida.

Por que análise de algoritmos ?

- Vamos ver que conseguimos saber muito sobre o comportamento de um algoritmo em termos de tempo de execução, espaço usado e qualquer outro recurso apenas fazendo análise
- É possível, por exemplo, rodar um experimento com entradas pequenas, entender o tempo e fazer estimativas para o comportamento com entradas maiores

EXEMPLO COM FIBONACCI

- Vamos olhar 3 programas que calculam a sequência de fibonacci (custos aproximados entre parênteses):
  - um recursivo ($0{,}45 \cdot 1{,}618^n$)
  - um que usa programação dinâmica (n)
  - um que calcula o valor usando uma fórmula pronta. (log n)

Por que análise de algoritmos ?

- O exemplo de fibonacci dado, que rodou em cerca de 80 segundos para a entrada de 50 elementos, rodaria em cerca de 63 mil anos para entrada 100.
- Rodaria em 1.788.455.772.335.113 anos para entrada 150!
- Isso dá cerca de 210 mil vezes a idade da terra!

Tempos para calcular fibonacci

Table

n	F. doubling	F. matrix	F. doubling, naïve	F. matrix, naïve	dynamic prog.	recursive
1	5 414	1 042	4 197	887	10	4
5	5 989	3 677	4 787	3 147	172	219
10	6 283	4 952	5 022	4 154	370	3 113
20	6 542	5 923	5 234	4 985	763	394 000
40	6 818	6 880	5 433	5 715	1 552	5 980 000 000
50	6 806	7 742	5 486	6 446	2 023	
100	7 279	9 225	5 904	7 717	4 472	
200	8 006	11 700	6 556	9 969	10 640	
1 000	14 660	50 870	13 130	48 230	118 000	
10 000	466 400	3 519 000	538 400	4 382 000	8 700 000	
100 000	20 800 000	130 600 000	51 640 000	330 700 000	819 700 000	
1 000 000	846 100 000	4 724 000 000	5 182 000 000	28 310 000 000	82 000 000 000	
3 981 072	7 654 000 000					

Escolhas menos dramáticas

- O exemplo de fibonacci é extremo e parece pouco útil.
- Muitas vezes, a troca de um algoritmo por um mais eficiente pode resultar em economia de tempo e dinheiro para empresas.
- Erros na escolha do algoritmo geram prejuizos.

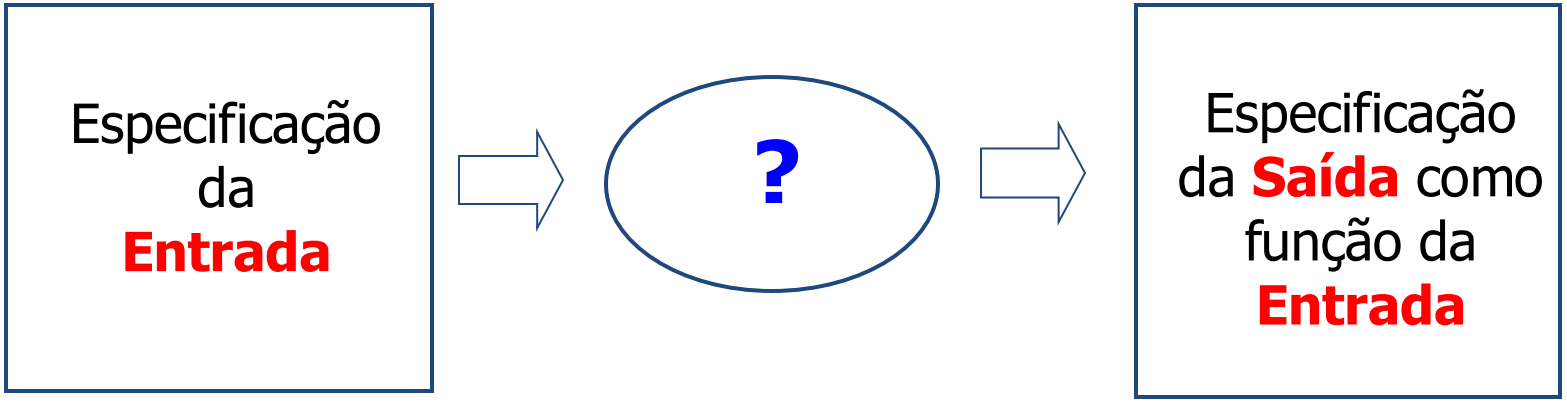
Por que projeto de algoritmos ?

- Aprendemos a solucionar problemas de maneira empírica
- Contudo, há certos padrões no desenvolvimento de soluções que podem ser repetidos e aplicados na resolução de novos problemas
- Uso de técnicas de projeto pode aumentar a velocidade para encontrar soluções, ajudar a resolver problemas que antes não saberíamos resolver ou ainda ajudar a encontrar soluções melhores

Algoritmos

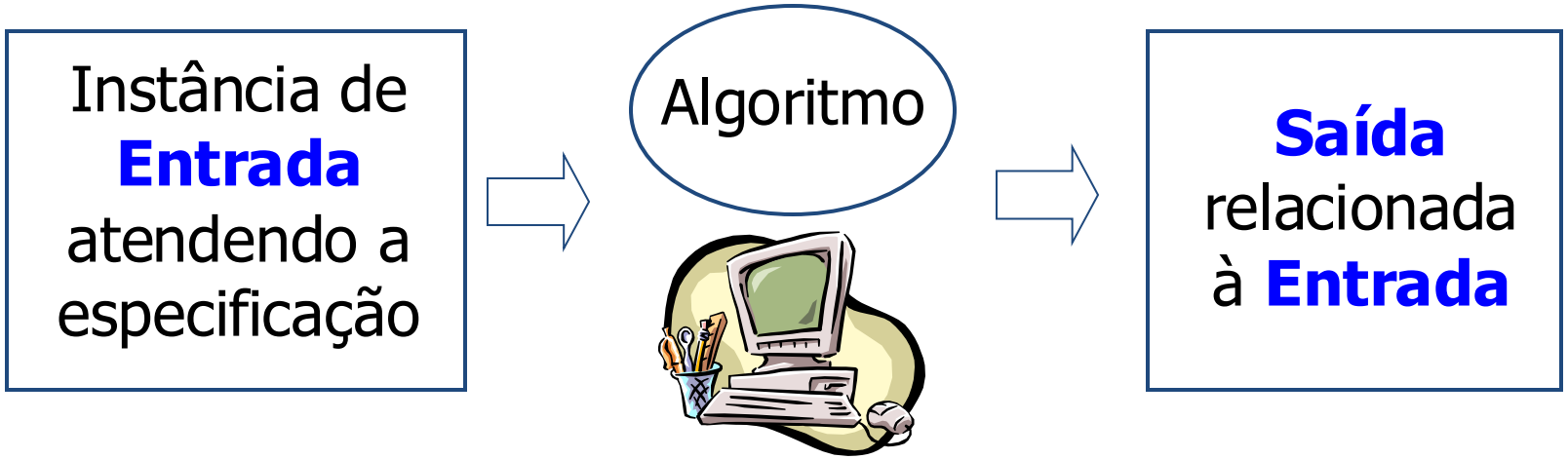
- **Um algoritmo**: A essência de um procedimento computacional composto por instruções seqüenciais passo a passo
- **Um programa**: Implementação de um algoritmo em uma dada linguagem de programação
- **Estrutura de dados**: forma de organizar os dados necessários à solução de um problema

Problema Algoritmico



- Existe um número infinito de *instâncias* da entrada que satisfazem a especificação. Por exemplo:
  - Uma sequência finita, ordenada, não decrescente de números naturais:
    - 1, 20, 908, 909, 100000, 1000000000.
    - 3.

Solução Algoritmica



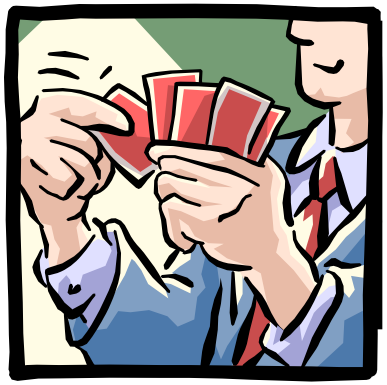
- O Algoritmo descreve ações sobre a instância de entrada
- Existem infinitos algoritmos corretos para um mesmo problema algorítmico

Examplo: Problema da Ordenação

ENTRADA
sequência de números

$a_1, a_2, a_3, \ldots, a_n$

2 5 4 10 7



SAÍDA
uma permutação da sequência de entradas

$b_1, b_2, b_3, \ldots, b_n$

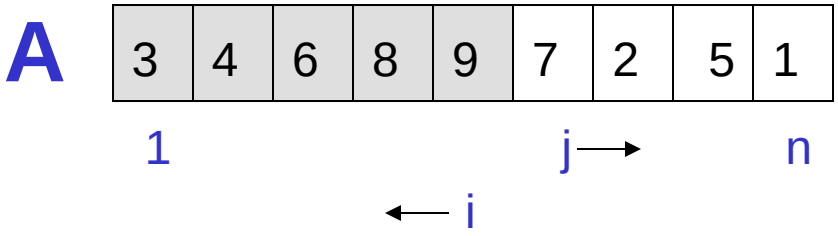
2 4 5 7 10

Corretude
Para qualquer entrada dada, o algortimo termina com saída

- $b_1, b_2, b_3, \ldots, b_n$, onde
- $b_1 < b_2 < b_3 < \ldots < b_n$

Tempo de Execução
Depende de:
- número de elementos (n)
- o quão (parcialmente) ordenada está a lista
- solução algoritmica

Ordenação por Inserção



"insira A[j] na sequência
ordenada A[1..j-1]"

```
para j=2 até n
    faça pivot:=A[j]
      i ← j-1
      enquanto i>0 e A[i]>pivot faça
        A[i+1] ← A[i]
        i--;
      fim-enquanto
      A[i+1] ← pivot;
```

Análise de Algoritmos

- Eficiência:
  - Tempo de execução
  - Espaço (memória) usado
- Eficiência como função do tamanho da entrada

Modelos de Computação

- O que deve ser contabilizado ?
- Modelo RAM
  - Instruções (considerando tempo constante):
    - Aritiméticas (+, -, *, etc.)
    - Atribuição
    - Comparação
    - Controle (desvios, chamadas de procedimento, etc.)

Análise: Ordenação por Inserção

- Determinar o **tempo de execução** como função do tamanho da entrada

```
para j=2 até n
    faça pivot:=A[j]
       i ← j-1

       enquanto i>0 e A[i]>pivot faça

          A[i+1] ← A[i]
          i--;

       fim-enquanto

       A[i+1] ← pivot;
```

n-1 vezes

Para cada uma das n-1 vezes, t_j vezes

Análise: Ordenação por Inserção

- Determinar o **tempo de execução** com função do tamanho da entrada

para j=2 **até** n	n
faça pivot:=A[j]	n-1
i ← j-1	n-1
enquanto i>0 **e** A[i]>pivot **faça**	$\sum_{j=2}^{n} t_j$
A[i+1] ← A[i]	$\sum_{j=2}^{n} (t_j - 1)$
i--;	$\sum_{j=2}^{n} (t_j - 1)$
fim-enquanto	
A[i+1] ← pivot;	n-1

Melhor/Pior/Médio

- **Melhor Caso**
  - elementos já ordenados
  - todos os testes falham, o loop interno nunca é executado
  - neste caso, t_j=1 e a operação é executada n vezes

$$\sum_{j=2}^{n} t_j = t_2 + t_3 + \ldots + t_n = 1 + 1 + \ldots + 1 = n - 1$$

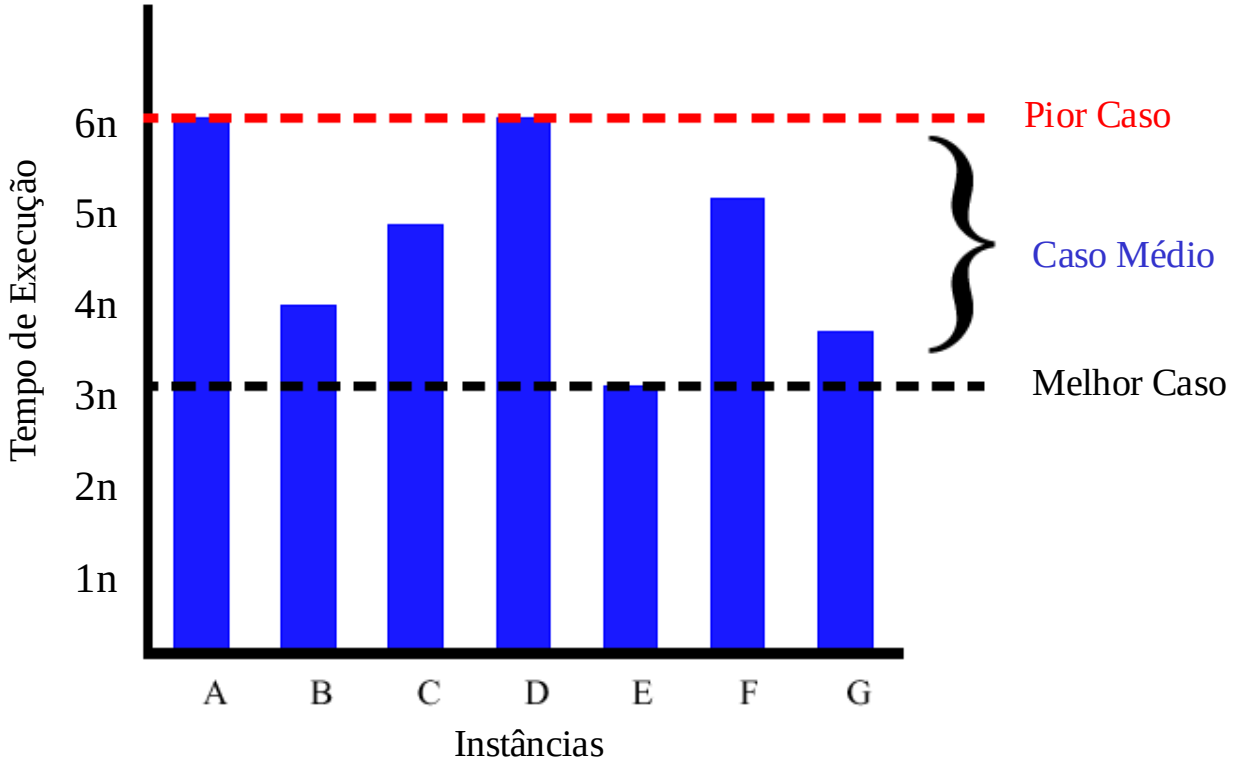
  - Portanto a complexidade de tempo é dada por f(n)=n-1

Melhor/Pior/Médio

- **Melhor Caso**
  - elementos já ordenados
  - $t_j=1$, $T(n) = C_1\, n$, ou seja, tempo linear.
- **Pior caso**: elementos em ordem inversa
  - $t_j=j$, $T(n) = C_2\, n^2$, ou seja, tempo quadrático
- **Tempo médio**:
  - $t_j=j/2$, $T(n) = C_2\, n^2$, ou seja, tempo quadrático

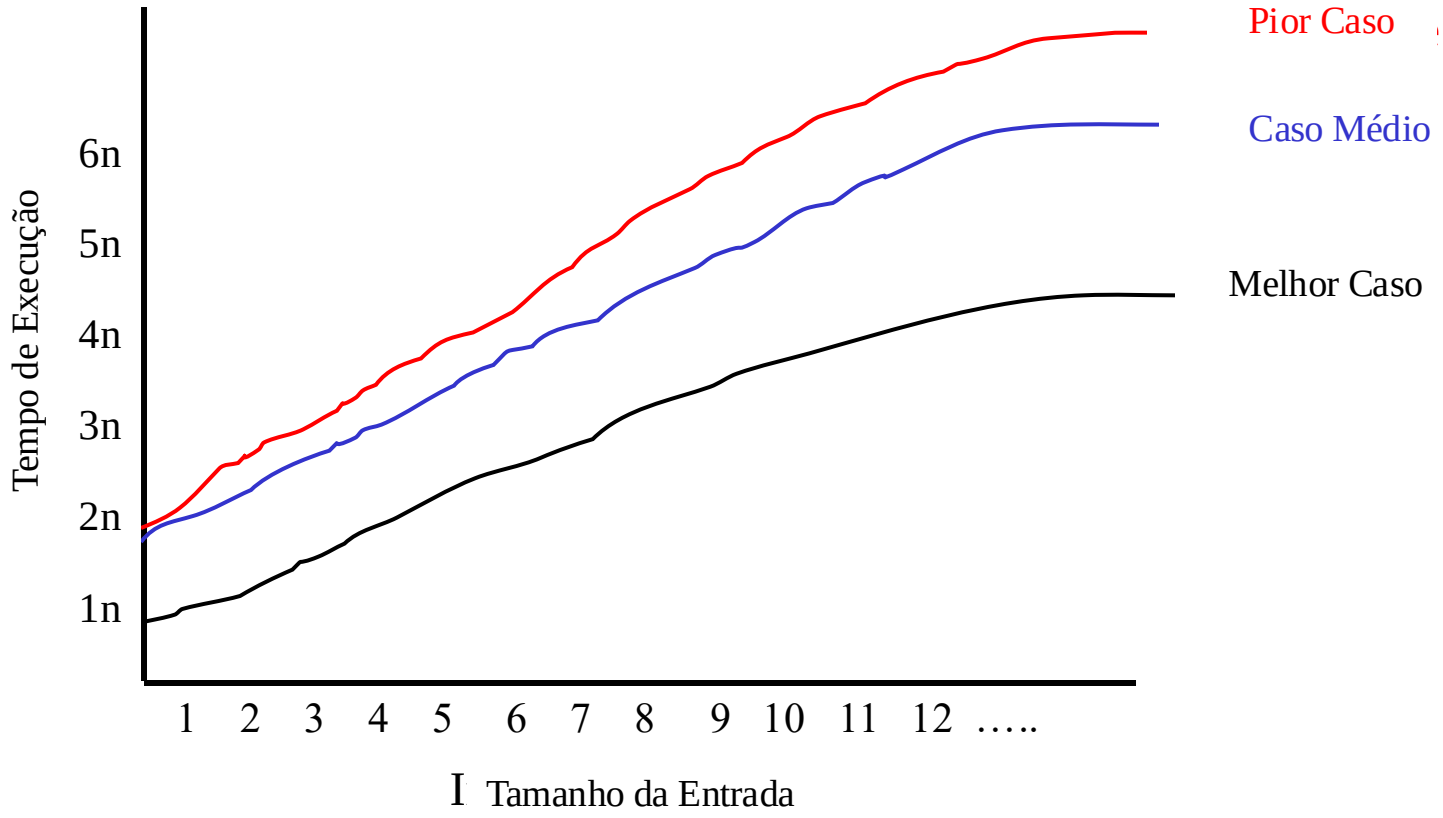
Melhor/Pior/Médio (2)

– Determinar o tempo de execução para diferentes instancias de tamanho n:



Melhor/Pior/Médio (3)

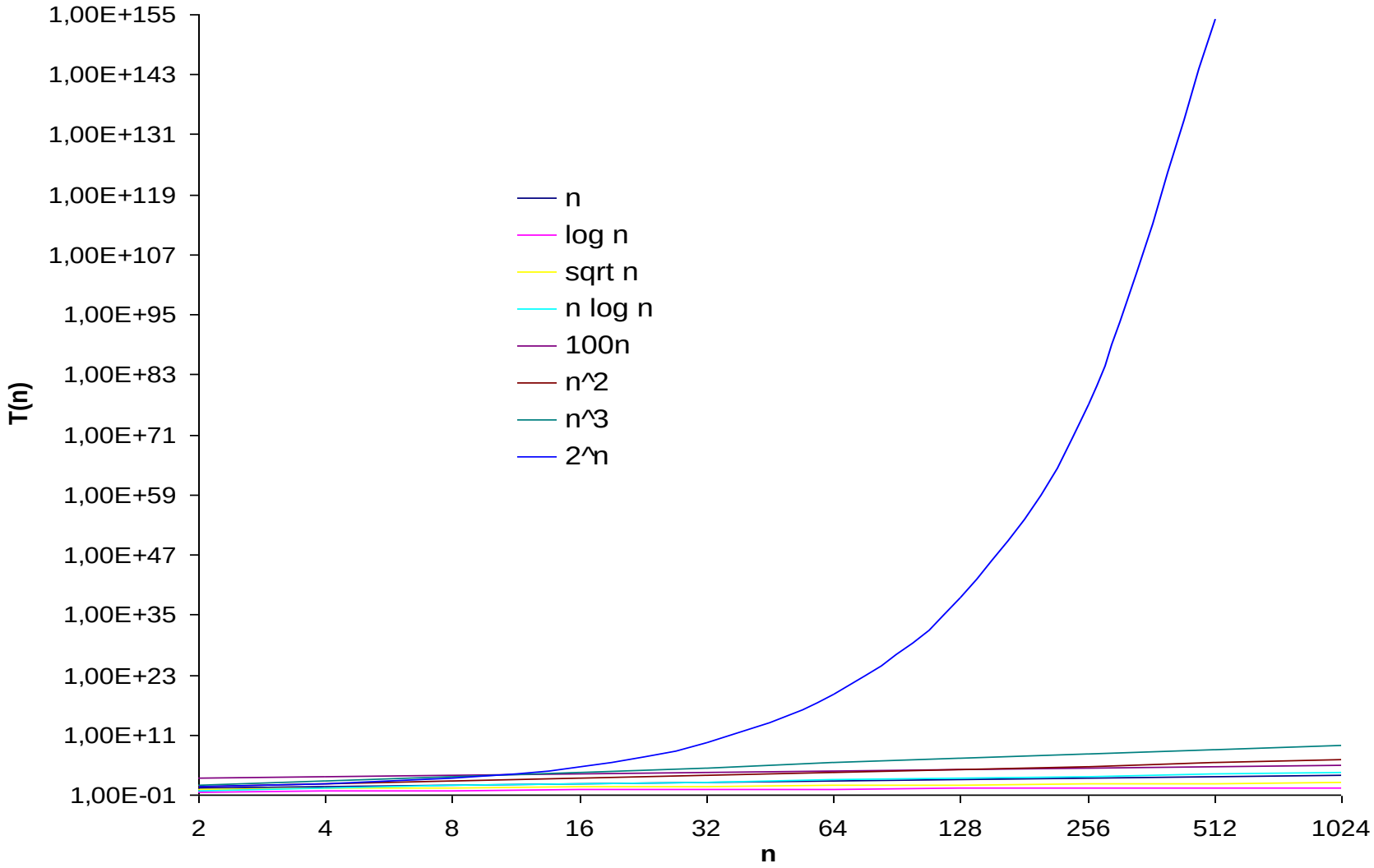
– Para instâncias de todos os tamanhos:



Melhor/Pior/Médio (4)

- **O pior caso** é geralmente usado:
  - Estabelece um limite superior na complexidade de tempo do algoritmo
  - Para alguns algoritmos o pior caso é bastante freqüente
  - Frequentemente o caso médio é tão ruim quando o pior caso
  - Encontrar o caso médio pode ser muito difícil

Funções de Crescimento (2)



O que é razoável em computação?

	função/ n	10	20	50	100	300
Polinomial	n^2	1/10,000 segundos	1/2,500 segundos	1/400 segundos	1/100 segundos	9/100 segundos
Polinomial	n^5	1/10 segundos	3.2 segundos	5.2 minutos	2.8 horas	28.1 dias
Exponencial	2^n	1/1000 segundos	1 segundos	35.7 anos	400 trilhões de séculos	Séculos de 75 dígitos
Exponencial	n^n	2.8 horas	3.3 trilhões de anos	Seculos de 70 dígitos	Séculos de 185 dígitos	Séculos de 728 dígitos

Exemplo 2: Busca

ENTRADA

- sequência de números
- um número (cosulta)

$a_1, a_2, a_3, \ldots, a_n;\ q$

2 5 4 10 7; 5

2 5 4 10 7; 9

SAÍDA

- índice do número or *NIL*

j

2

NIL

Busca (2)

```
j=1
while j<=length(A) and A[j]!=q
   do j++
if j<=length(A) then return j
else return NIL
```

- Pior caso: *f(n)=n*, caso-médio: *n/2*
- Não se pode fazer melhor ! Este é um limite inferior para o problema de busca em uma seqüência arbritrária.

Exemplo 2: Busca

ENTRADA

- sequência de números ordenada não-descendente
- um número (cosulta)

$a_1, a_2, a_3, \ldots, a_n;\ q$

2 4 5 7 10; 5

2 4 5 7 10; 9

SAÍDA

- o índice do número encontrado ou *NIL*

j

2

NIL

Busca binária

- Idéia: Divisão e conquista, uma das técnicas chaves para projeto de algoritmos

```
left=1
right=length(A)
do
   j=(left+right)/2
   if A[j]==q then return j
   else if A[j]>q then right=j-1
   else left=j+1
while left<=right
return NIL
```

Busca Binária – Análise

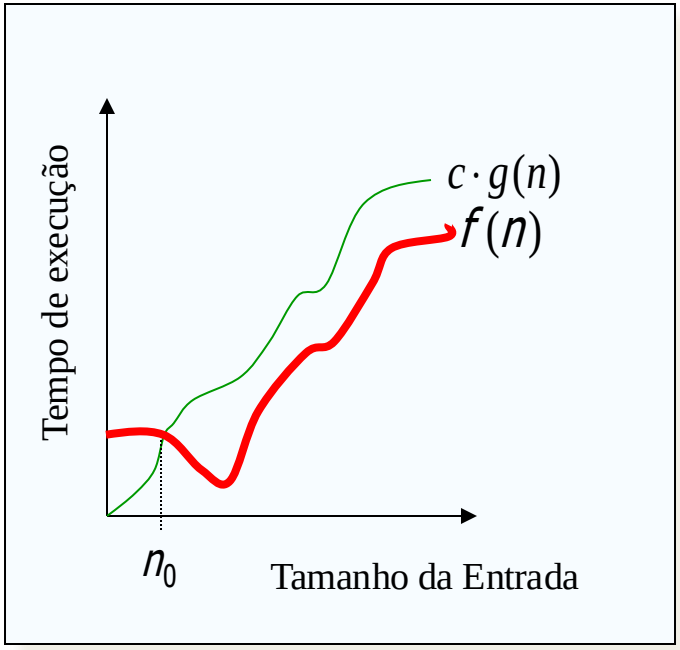
- Quantas vezes o laço é executado ?
  - A cada interação o número de posições n é cortado ao meio
  - Quantas vezes se corta ao meio n para se chegar a 1?
  - *lg n*

Análise Assintótica

- **Objetivo**: simplificar a análise de complexidade de tempo eliminando "detalhes" que podem ser afetados por fatores de implementação ou hardware
  - $3n^2 \rightarrow n^2$
- **Comportamento assintótico de uma função**: como sua imagem cresce com o tamanho da entrada.
- Exemplo: É comum querermos saber como o tempo de execução do algoritmo cresce com o tamanho da entrada.

Notação Assintótica

- *Notação O (O* - Notation)
  - Limite assintótico superior
  - *f(n) = O(g(n))*, se existem constantes positivas *c* e n_0, **f(n) ≤ c g(n)** para $n \geq n_0$
  - *f(n)* e *g(n)* são funções sobre inteiros não negativos
- Usada para análise do pior caso



Notação O

$$f(n) \pounds cg(n)$$

$$c \geq \frac{f(n)}{g(n)}$$

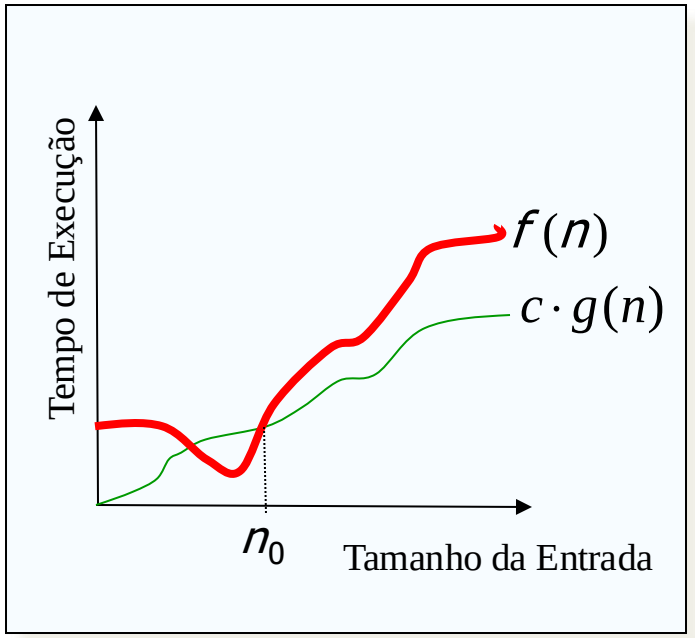
$$\lim_{n \to \infty} \frac{f(n)}{g(n)}$$

RESULTADOS POSSÍVEIS:

- *0: siginifica que constantes c e n_0 existem!*
- *Constante > 0: siginifica que constantes c e n_0 existem!*
- *Infinito: singifica que não é possível atender à inequação (razão entre as funções cresce sempre e f(n) não pode ser limitada por c.g(n), sendo c constante!*

Notação Assintótica (2)

- Notação Ω
  - limite assintótico inferior
  - $f(n) = \Omega(g(n))$ se existem constantes positivas c e n_0, *tais que* **c g(n) ≤ f(n)** para $n \geq n_0$
- Usada para descrever o melhor caso de tempos de execução ou limites inferiores de problemas algorítmicos
  - P.ex., o limite inferior para busca em lista não ordenada é $\Omega(n)$.

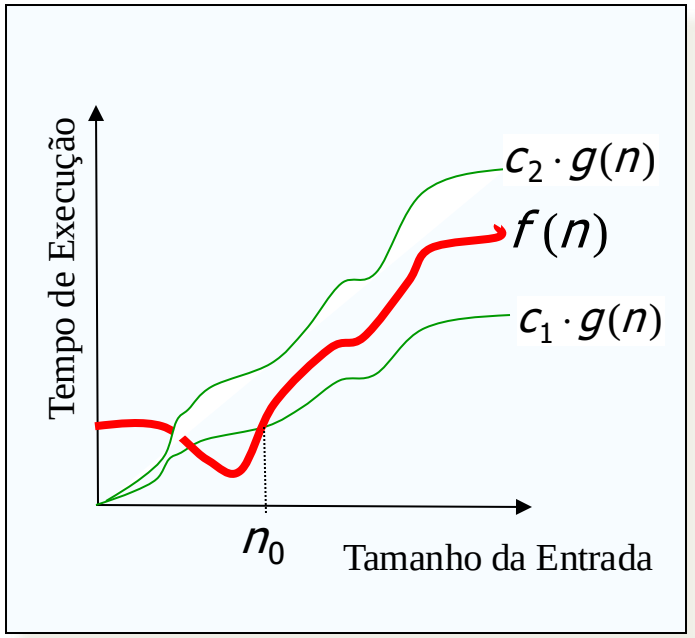


Notação Assintótica (3)

- Regra prática: Remover termos de menor ordem e fatores constantes.
  - $50\ n \log n$ é $O(n \log n)$
  - $7n - 3$ é $O(n)$
  - $8n^2 \log n + 5n^2 + n$ é $O(n^2 \log n)$
- Nota: Mesmo que $50\ n \log n$ seja $O(n^5)$, costuma-se utilizar aproximações de menor ordem possível !!!

Notação Assintótica (4)

- Notaçao Θ
  - Limite assintótico máximo e mínimo
  - $f(n) = \Theta(g(n))$ se existem constantes c_1, c_2, e n_0, *tais que* $\mathbf{c_1\, g(n) \leq f(n) \leq c_2\, g(n)}$ para $n \geq n_0$
- $f(n) = \Theta(g(n))$ *se, e somente se,* $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$
- $O(f(n))$ é frequente confundido com $\Theta(f(n))$



Notação Assintótica (5)

- Notação *o* : *f(n)=o(g(n))*
  - Para todo *c positivo,* deve existir n_0, tal que **f(n)** $\leq$ **c g(n)** para $n \geq n_0$
  - Usada para comparação de tempos de execução. Se *f(n)=o(g(n)),* dizemos que *g(n) domina f(n).*
- Notação ω:*f(n)=*ω*(g(n))*
  - Análoga a função *o* com relação à Ω

Notação Assintótica (6)

- Analogia com número reais
  - $f(n) = O(g(n)) \cong f \leq g$
  - $f(n) = \Omega(g(n)) \cong f \geq g$
  - $f(n) = \Theta(g(n)) \cong f = g$
  - $f(n) = o(g(n)) \cong f < g$
  - $f(n) = \omega(g(n)) \cong f > g$
- Abuso de notação:
  - $f(n) = O(g(n))$ indica que $f(n) \in O(g(n))$

Observações

- Log n é sempre o(n^b) se b é positivo. Prova-se usando-se a regra de l'Hôpital onde limite f/g é igual ao limite de f'/g' se f e g convergem para o infinito.
- Traduzindo: log cresce pouquíssimo!

$$\lim_{n \to \infty} \frac{\log n}{n^b} = \lim_{n \to \infty} \frac{\frac{1}{n}}{bn^{b-1}} = \lim_{n \to \infty} \frac{1}{bn^b} = 0$$

Exercícios

- Verifique se as afirmativas são corretas:
- No pior caso, o algoritmo de inserção é

$$\Theta(n^2)$$

$$2^{2n} = O(2^n)$$

$$2^{n+1} = O(2^n)$$

$$O(n^2) + O(n^2) = O(n^2)$$

$$O(n) \times O(n) = O(n)$$